컴퓨터시스템프로그래밍 - 09/15 강의 퀴즈

범위: 슬라이드 07 `Code Optimization and Linking` 및 09/15 강의 전사본

이름: _____ 학번: _____ 풀이일: _____

풀이 원칙: C와 AT&T x86-64 표기법을 기준으로 답하시오. 필요한 곳에서는 "왜 안전한가/안전하지 않은가"를 함께 설명하시오.

Part 1. OX (5문항)

1. O / X 컴파일러 최적화는 보통 알고리즘의 점근적 복잡도를 바꾸기보다, 같은 복잡도 안에서 상수 인자와 실행 비용을 줄이는 데 초점을 둔다.

답안

2. O / X 이 강의에서 말하는 global optimization은 여러 함수 전체를 함께 분석하는 inter-procedural optimization과 같은 뜻이다.

답안

3. O / X `long mask = 0xFF << 8;`을 `long mask = 0xFF00;`으로 바꾸는 것은 constant folding의 예이다.

답안

4. O / X C에서 `a > 0`, `b < 0`일 때 `a - b > 0`은 실제 기계의 2의 보수 연산 결과만 고려하면 항상 참이므로, signed overflow를 고려할 필요가 없다.

답안

5. O / X 정적 라이브러리(`.a`)는 관련 relocatable object file들을 아카이브로 묶고, 링커는 해결되지 않은 external reference를 해결하는 멤버를 필요할 때 실행 파일에 포함한다.

답안

Part 2. 서술형 (5문항)

6. C 소스 파일이 실행 파일이 되기까지의 흐름을 preprocessing, compilation, assembling, linking 순서로 설명하시오. 각 단계의 입력·출력과, 왜 링크 이전의 object file이 relocatable해야 하는지를 포함하여 5–8문장으로 쓰시오.

답안

7. local optimization과 global optimization의 분석 범위를 basic block 및 control-flow graph 관점에서 비교하시오. 여기서 global이 여러 함수를 동시에 본다는 뜻이 아닌 이유와, inter-procedural analysis가 더 어려운 이유까지 5–8문장으로 설명하시오.

답안

8. constant folding, strength reduction, dead code elimination, CSE를 각각 한 문장씩 정의하고, 네 변환 모두가 프로그램의 관찰 가능한 동작을 바꾸면 안 되는 이유를 5–8문장으로 설명하시오.

답안

9. `lower1`**의** `strlen(s)`**를 루프 밖으로 옮긴** `lower2`**가 사람에게는 자연스러워 보여도, 컴파일러가 일반적으로 자동 변환하기 어려운 이유를 설명하시오. loop invariant의 조건, 함수 호출의 부작용/별칭 가능성, whole-program 또는 inter-procedural 분석 비용을 포함하여 5–8문장으로 쓰시오.**

답안

10. `main.c`**가** `sum(array, 2)`**를 호출하고** `sum.c`**가** `sum`**을 정의할 때, symbol table construction, symbol resolution, relocation이 각각 무엇을 기록하거나 해결하는지 설명하시오.** `main.c`**에서의** `sum`**,** `sum.c`**에서의** `sum`**, 전역** `array`**를 정확히 구분하여 5–8문장으로 쓰시오.**

답안

Part 3. 적용·계산형 (10문항)

11. 다음 코드를 더 단순하고 빠른 형태로 바꾸고, 각 변환의 이름을 쓰시오.

```
long f(long b) {
    long mask = 0xFF << 8;
    long a = b * 5;
    return a + mask;
}
```

`b * 5`에 대해 곱셈을 사용하지 않는 동치식을 제시하고, constant folding과 strength reduction이 각각 어느 줄에 적용되는지 설명하시오.

답안

12. 다음 코드를 CSE 관점에서 다시 작성하시오. `v[i]`**의 주소 계산과 구조체 멤버 접근을 반복하지 않도록 하되, 원래와 같은 값을 계산해야 한다. 이후 메모리 접근·주소 계산을 줄일 수 있는 이유를 설명하시오.**

```
typedef struct { long x, y; } vec_t;
long norm[100];
vec_t v[100];

void set_norm(long i) {
```

```
    norm[i] = v[i].x * v[i].x + v[i].y * v[i].y;
}
```

답안

13. 다음 `pred`를 `func` 안으로 inlining한 뒤, constant folding과 dead code elimination을 적용한 최종 C 코드를 작성하시오. 함수 호출 제거가 왜 이득이 될 수 있는지, 코드 크기가 늘어날 수 있는 반대 비용도 한 가지씩 쓰시오.

```
int pred(int x) {
    if (x == 0) return 0;
    else return x - 1;
}

int func(int y) {
    return pred(y) + pred(0) + pred(y + 1);
}
```

답안

14. 다음 두 반복문 중 `n * i`를 루프 밖으로 옮길 수 있는 것을 고르고, 옮긴 코드를 작성하시오. 옮길 수 없는 경우에는 루프 불변식이 깨지는 이유를 쓰시오.

```
// A
for (long j = 0; j < n; j++)
    a[n * i + j] = b[j];

// B
for (long j = 0; j < n; j++) {
    i += b[j];
    a[n * i + j] = b[j];
}
```

답안

15. 다음 루프를 4-way loop unrolling으로 변환하시오. `nelts`**가 4의 배수가 아닐 때도 모든 원소를 처리하는 cleanup loop를 포함해야 한다. branch overhead, 코드 크기, cache 또는 instruction cache 관점의 trade-off를 설명하시오.**

```
for (size_t i = 0; i < nelts; i++)
    A[i] = B[i] * k + C[i];
```

답안

16. 다음 두 함수의 시간 복잡도를 각각 `n = strlen(s)`**로 나타내고, 차이가 나는 이유를 설명하시오.** `lower1`**을 사람이** `lower2`**처럼 고치는 것이 유리할 수 있는 이유와, 컴파일러가 항상 이를 대신하지 못하는 이유를 구분하시오.**

```
void lower1(char *s) {
    for (size_t i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z') s[i] -= ('A' - 'a');
}

void lower2(char *s) {
    size_t i, n = strlen(s);
    for (i = 0; i < n; i++)
        if (s[i] >= 'A' && s[i] <= 'Z') s[i] -= ('A' - 'a');
}
```

답안

17. 다음 함수에서 `a == INT_MAX`, `b == -1`일 수 있다고 하자. 소스 수준에서 어떤 연산이 문제가 되는지, 왜 최적화된 컴파일러가 안쪽 조건을 제거할 수 있는지, 그리고 실행 결과를 특정한 2의 보수 값으로 단정하면 안 되는 이유를 설명하시오.

```
int g(int a, int b) {
    if (a > 0) {
        if (b < 0) {
            if ((a - b) > 0)
                return 1;
        }
    }
    return 0;
}
```

답안

18. 다음은 아직 링크되지 않은 `main.o`의 일부이다. 각 relocation entry가 어느 심볼을 위한 것인지 쓰고, `%edi`와 `%esi`에 각각 어떤 C 수준의 인자가 전달되는지 설명하시오. 링크 후 `sum`의 시작 주소가 `0x4004e8`이고 다음 명령어 주소가 `0x4004e3`이면, `callq`의 PC-relative displacement가 왜 `5`인지 계산하시오.

```
0:  48 83 ec 08          sub    $0x8,%rsp
4:  be 02 00 00 00       mov    $0x2,%esi
9:  bf 00 00 00 00       mov    $0x0,%edi
   R_X86_64_32 array
e:  e8 00 00 00 00       callq  13 <main+0x13>
   R_X86_64_PC32 sum-0x4
```

답안

19. 다음 두 파일의 심볼을 `global definition`, `local definition`, `external reference`로 분류하시오. 특히 `helper`, `shared`, `sum`이 각 object file에서 어떻게 보이는지 표로 정리하고, `static`이 붙은 함수가 다른 파일의 호출 대상이 될 수 없는 이유를 쓰시오.

```
/* main.c */
int sum(int *a, int n);
int shared = 1;
static int helper(int x) { return x + shared; }
int main(void) { int a[2] = {1, 2}; return sum(a, 2) + helper(0); }
```

```
/* sum.c */
int sum(int *a, int n) { return a[0] + a[1]; }
```

답안

20. 다음 요구를 만족하는 명령어와 실행 시 기대 결과를 쓰시오.

- `addvec.c`, `multvec.c`를 묶은 정적 라이브러리 `libvector.a`를 만든다.
- `main2.o`와 위 라이브러리를 사용해 정적으로 링크된 `prog2c`를 만든다.
- 별도의 동적 실행 파일 `prog`가 요구하는 shared library를 확인한다.

각 단계에 적절한 `ar`, `gcc`, `ldd` 명령을 쓰고, 정적 라이브러리 아카이브·정적 링크·동적 의존성 확인의 역할을 구분하시오.

답안

Part 4. 흔한 함정 (3문항)

21. Signed overflow 함정. 다음과 같은 "오버플로우 검출" 코드를 최적화 빌드에서 신뢰해도 되는지 판정하고, 안전한 설계 방향을 쓰시오.

```
int add_overflowed(int x) {
    return (x + 1 < x);
}
```

답에는 signed overflow의 언어 의미, 컴파일러가 할 수 있는 추론, 범위 검증 또는 적절한 도구/타입을 이용한 대안을 포함하시오.

답안

22. Linkage 함정. 다른 파일에서 호출해야 하는 `sum`의 정의를 실수로 `static int sum(...)`으로 바꾸었다. `main.c`에는 이전과 같이 `int sum(...);` 선언과 호출이 있다. 링크 시 어떤 종류의 문제가 생기는지, `static`을 언제 써야 하는지, 심볼 공간 관점에서 수정안을 설명하시오.

답안

23. 안전하지 않은 code motion 함정. 다음 루프에서 `unknown(s)`를 무조건 루프 밖으로 옮기는 최적화가 왜 위험한지 설명하시오. 안전하려면 어떤 사실을 증명해야 하는지, 함수의 side effect와 inter-procedural 분석의 관계를 포함하시오.

```
for (size_t i = 0; i < n; i++) {
    total += unknown(s);
    s[i] = 0;
}
```

답안

정답 및 해설

Part 1

1. O. 컴파일러 최적화는 보통 알고리즘 자체를 바꾸지 않고, instruction 수·메모리 대기·branch 비용 같은 상수 인자를 줄인다.

2. X. 이 강의의 global optimization은 한 함수의 control-flow graph 전체를 다룬다. 여러 함수의 관계를 함께 보는 것은 inter-procedural analysis다.

3. O. 두 피연산자가 compile-time constant이므로 컴파일러가 실행 전에 `0xFF00`을 계산할 수 있다.

4. X. 수학적으로는 양수처럼 보여도 `a-b`가 signed overflow하면 C에서 정의되지 않은 동작이다. 컴파일러는 overflow가 발생하지 않는 경로를 전제로 조건을 참으로 최적화할 수 있다.

5. O. 정적 라이브러리는 object file 아카이브이며, 링커는 필요한 unresolved symbol을 해결하는 멤버를 가져와 링크한다.

Part 2

6. 예시 핵심: preprocessing은 `#include` 등을 해소해 확장된 C 소스를 만든다. compilation은 C를 assembly로 변환한다. assembling은 assembly를 relocatable object file로 만든다. linking은 여러 object file과 필요한 라이브러리를 결합해 executable object file을 만든다. 별도 컴파일된 파일은 최종 배치 주소를 모르므로 참조와 주소를 나중에 고칠 수 있어야 한다. 이 재배치 가능성이 relocation의 출발점이다.

7. 예시 핵심: local optimization은 하나의 basic block 안에서 constant folding·local CSE 등을 수행한다. basic block은 single entry와 single exit를 갖는 최소 control-flow 단위다. global optimization은 한 함수의 control-flow graph와 loop nest를 대상으로 code motion·global CSE·dead code elimination 등을 수행한다. 따라서 여기서 global은 여러 함수 전체를 뜻하지 않는다. 함수 간 분석은 가능한 호출 대상, 별칭, side effect, 입력을 더 넓게 추적해야 하므로 비용과 복잡도가 커진다.

8. 예시 핵심: constant folding은 상수식의 결과를 미리 계산한다. strength reduction은 비싼 연산을 더 싼 동치 연산으로 바꾼다. dead code elimination은 절대 실행되지 않거나 결과가 덮어써지는 계산을 제거한다. CSE는 같은 부분식을 한 번 계산해 재사용한다. 네 변환 모두 동일한 입력에서 관찰 가능한 출력과 side effect를 보존해야 한다. 그렇지 않다면 최적화가 아니라 프로그램 의미 변경이다.

9. 예시 핵심: code motion은 옮길 식이 반복마다 같은 결과를 내는 loop invariant일 때만 안전하다. 호출의 정의나 side effect를 현재 compilation unit만으로 완전히 모를 수 있다. 포인터 별칭 때문에 호출 또는 루프 본문이 관련 메모리를 바꿀 가능성도 고려해야 한다. 모든 함수를 함께 분석하면 더 많은 사실을 알 수 있지만 inter-procedural 분석 비용이 크다. 사람이 `strlen`과 문자열 변경의 관계를 확인한 뒤 고치는 것과 컴파일러가 일반적인 C 함수 호출에 대해 증명하는 것은 다르다.

10. 예시 핵심: assembler는 각 object file에 자신이 정의하거나 참조하는 함수·전역 변수의 symbol table을 만든다. `main.c`에서 `sum`은 external reference이고, `sum.c`에서 `sum`은 global definition이다. `array`는 `main.c`가 정의한 전역 심볼이며 `sum` 호출의 첫 인자로 전달된다. linker는 symbol resolution으로 참조 `sum`을 정의 `sum`에 연결한다. relocation은 최종 executable의 section과 심볼 주소가 정해진 뒤 참조 위치를 고친다.

Part 3

11. 예시 답:

```
long f(long b) {
    long mask = 0xFF00;          // constant folding
    long a = (b << 2) + b;       // strength reduction
    return a + mask;
}
```

`0xFF << 8`은 상수 입력만 가지므로 compile time에 접을 수 있다. `5b = 4b+b`이고, `4b`는 `b << 2`로 표현할 수 있다.

12. 예시 답:

```
void set_norm(long i) {
    vec_t *elem = &v[i];
    long x = elem->x;
    long y = elem->y;
    norm[i] = x * x + y * y;
}
```

`v[i]` 주소 계산과 멤버 dereference를 재사용한다. 실제 생성 코드는 aliasing·register allocation을 고려하지만, 공통 부분식을 명시적으로 제거한 형태다.

13. 예시 답:

```
int func(int y) {
    int tmp = 0;
    if (y != 0) tmp = y - 1;
    if (y != -1) tmp += y;
    return tmp;
}
```

`pred(0)`은 0으로 접히고, 불필요한 `+0`과 항상 참인 분기는 제거된다. call/return, return address, 인자 전달 비용을 줄일 수 있다. 반대로 함수 본문을 여러 호출 지점에 복사하면 code size가 커져 instruction cache와 디버깅에 불리할 수 있다.

14. A만 가능하다. `n`과 `i`가 루프 동안 바뀌지 않으므로 다음처럼 쓴다.

```
long ni = n * i;
for (long j = 0; j < n; j++)
    a[ni + j] = b[j];
```

B는 매 반복에서 `i`를 변경하므로 `n*i`가 loop invariant가 아니다.

15. 예시 답:

```
size_t i = 0;
```

```
for (; i + 3 < nelts; i += 4) {
    A[i]   = B[i]   * k + C[i];
    A[i+1] = B[i+1] * k + C[i+1];
    A[i+2] = B[i+2] * k + C[i+2];
    A[i+3] = B[i+3] * k + C[i+3];
}
for (; i < nelts; i++)
    A[i] = B[i] * k + C[i];
```

루프 제어·branch의 빈도를 낮추고 CSE나 code motion 기회를 늘릴 수 있다. 반면 code size가 커지며 instruction cache 압박 때문에 항상 빨라지는 것은 아니다.

16. `lower1`**은 매 반복** `strlen`**이 최대 O(n)이므로 전체 O(n^2)이고,** `lower2`**는** `strlen`**을 한 번만 호출하므로 O(n)이다. 사람이 문자열 길이가 변하지 않음을 확인했다면** `n`**을 미리 저장하는 것이 유리하다. 그러나 컴파일러는 일반 함수 호출, 별칭, side effect를 보수적으로 판단해야 하므로 동일한 변환을 항상 보장할 수 없다.**

17. `INT_MAX - (-1)`**은 signed overflow를 일으킨다. C에서 그 연산의 동작은 정의되지 않았으므로 최적화기는 overflow가 발생하지 않는 유효한 실행만 전제로** `a-b > 0`**을 참으로 볼 수 있다. 따라서 안쪽 조건을 제거할 수 있다. 하드웨어가 우연히 2의 보수 wraparound를 보인다고 해서 이 C 프로그램의 결과를 이식성 있게 단정할 수는 없다.**

18. `R_X86_64_32 array`**는** `%edi`**에 들어갈 첫 번째 인자** `array`**의 주소를 고치는 entry이고,** `R_X86_64_PC32 sum-0x4`**는** `callq`**의** `sum` **참조를 고치는 entry다.** `%esi`**에는 두 번째 인자** `2`**가 들어간다. x86-64 System V 호출 규약에서 첫 두 정수/포인터 인자는** `%rdi`**,** `%rsi` **계열이다. displacement는 target - next-PC이므로** `0x4004e8 - 0x4004e3 = 5`**다.**

19. `main.o`**에서** `shared`**와** `main`**은 global definition,** `helper`**는 local definition,** `sum`**은 external reference다.** `sum.o`**에서** `sum`**은 global definition이다.** `helper`**는** `static`**이므로** `main.c` **compilation unit 내부에서만 보이며 external symbol로 제공되지 않는다. 따라서 다른 object file은** `helper`**를 해결할 수 없다.**

20. 예시 답:

```
ar rcs libvector.a addvec.o multvec.o
gcc -static -o prog2c main2.o -L. -lvector
ldd prog
```

`ar`는 object file들을 인덱스가 있는 archive로 묶는다. `gcc -static`은 필요한 라이브러리 코드를 실행 파일에 포함하도록 링크한다. `ldd`는 동적으로 링크된 `prog`가 load-time에 요구하는 shared library를 확인한다.

Part 4

21. 이 코드는 안전한 signed overflow 검출식이 아니다. `x == INT_MAX`**에서** `x+1` **자체가 signed overflow이며 C 의미론에서는 정의되지 않은 동작이다. 따라서 최적화기는 비교가 항상 거짓이라고 단순화할 수 있다. 덧셈 전 범위를 검사하거나, 요구사항에 맞는 unsigned 연산·컴파일러 내장 overflow 검사 기능을 사용해야 한다.** `-fwrapv`**는 GCC의 특정 의미론 선택이지, 임의의 C 코드가 안전하다는 일반 보장은 아니다.**

출처: GCC, *Code Generation Options* (`-fwrapv`, `-fstrict-overflow`)

https://gcc.gnu.org/onlinedocs/gcc/Code-Gen-Options.html

22. `sum.c`**의** `static sum`**은 local definition이므로 다른 object file에 export되지 않는다.** `main.o`**에는 external reference** `sum`**이 남고, linker는 이를 해결할 전역 정의를 찾지 못해 undefined reference 오류를 낸다. 여러 파일에서 호출해야 하는 API에는** `static`**을 붙이지 않고 선언과 정의의 타입을 일치시킨다. 반대로 파일 내부 구현 세부사항만 사용하는 함수·전역 변수에는** `static`**을 사용해 external symbol table을 불필요하게 오염시키지 않는다.**

출처: SEI CERT C, *DCL19-C. Minimize the scope of variables and functions*

https://wiki.sei.cmu.edu/confluence/display/c/DCL19-C.+Minimize+the+scope+of+variables+and+functions

23. `unknown(s)`**가** `s` **또는** `s`**가 가리키는 메모리를 읽거나 바꾸지 않는다는 보장이 없으므로, 호출을 한 번만 수행하도록 밖으로 옮기면 호출 횟수와 결과가 달라질 수 있다. 루프 본문도** `s[i] = 0`**으로 관련 메모리를 바꾼다. 안전한 code motion을 하려면 매 반복의** `unknown(s)` **결과가 같고 호출의 관찰 가능한 side effect가 없음을 증명해야 한다. 이 증명에는 함수의 pure/const 성질과 함수 경계를 넘는 modification/reference 분석이 필요할 수 있다.**

출처: GCC, *Optimize Options* (`-fipa-pure-const`, `-fipa-modref`)

https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html